Software Requirements Specification (SRS)

NexusChat — Real-Time Community and Collaboration Platform

Version 1.0

PES UNIVERSITY, BENGALURU
Department of Computer Science and Engineering
UE24CS341A — Software Engineering
Academic Year 2026
Prepared by:
Student 1 — SRN
Student 2 — SRN
Student 3 — SRN
Student 4 — SRN

Revision History

Version	Date	Author	Reason
1.0	2026-09-01	Project Team	Initial Agile baseline

1. Introduction

1.1 Purpose

This SRS defines the functional, non-functional, interface, security and domain requirements for NexusChat, a real-time browser chat application using WebSockets.

1.2 Intended Audience and Reading Suggestions

The document is intended for student developers, testers, project evaluators, project managers/product owners and future maintainers.

1.3 Product Scope

NexusChat enables authenticated users to participate in persistent communities and channels and exchange messages in real time. It provides permission-controlled administration, direct messaging, presence, notifications, reactions, search and moderation.

1.4 References

Primary academic reference: PES University, Department of Computer Science and Engineering, UE24CS341A Software Engineering Project Guidelines 2026. The guideline requires Agile methodology, active GitHub/Jira-like tracking, CI/CD, code reviews and security validation. filecite turn0file0 L13-L27

2. Overall Description

2.1 Product Perspective

NexusChat is a new self-contained web application. The browser communicates with the application server using HTTPS for request/response operations and WebSockets for real-time events. A relational database stores durable state.

2.2 Product Functions

- User registration, login/logout and session/token management.
- Profile management including display name, avatar reference and status.
- Community/server creation, membership and invitation handling.
- Text channel creation, update and permission-aware deletion.
- Real-time channel messaging over WebSockets.
- Direct one-to-one messaging.
- Online/idle/offline presence and typing indicators.
- Message reactions, editing and deletion.

- Role-based permissions and moderation actions.
- Unread counts, notifications and message search.
- Admin/moderator audit events.
- Responsive browser UI and error handling.

2.3 User Classes and Characteristics

User Class	Characteristics	Access
Guest	Unauthenticated visitor	Registration/login
Member	Normal authenticated user	Profile, communities, channels, messages, DMs
Moderator	Trusted community member	Member capabilities plus moderation
Community Admin	Owner/administrator	Community/channel/role management
System Maintainer	Development/deployment role	Operational maintenance

2.4 Operating Environment

- Modern Chromium/Firefox/Safari browser
- Node.js LTS server
- PostgreSQL/MySQL-compatible relational database
- HTTPS and WSS in deployment
- Docker and Jenkins

2.5 Design and Implementation Constraints

- Agile methodology is mandatory. 【filecite】turn0file0【L18-L24】
- GitHub and Jira-like backlog tracking must be actively used. 【filecite】turn0file0【L20-L22】
- CI/CD, code reviews and security validation are mandatory. 【filecite】turn0file0【L21-L22】
- The prescribed chat case study uses WebSockets for instant messaging. 【filecite】turn0file0【L148-L154】
- Every member must own a complete functionality and contribute implementation work. 【filecite】turn0file0【L39-L40】

2.6 Assumptions and Dependencies

- Reliable network connectivity
- Supported database and WebSocket library
- Available GitHub repository and Agile board
- Evaluator accepts the Node.js/WebSocket stack
- No voice/video subsystem in MVP

3. External Interface Requirements

3.1 User Interfaces

- Registration/login
- Community sidebar
- Channel list and message timeline
- DM view
- Member list with presence
- Administration panels
- Search/notification views
- Validation/error messages

3.2 Software Interfaces

Interface	Purpose	Data
REST API	Auth, profiles, communities, channels, search, moderation	JSON
WebSocket API	Messages, presence, typing, reactions	Versioned JSON events
Relational DB	Durable state	Users, communities, channels, messages, roles, audit
Jenkins	CI/CD	Build/test/analysis/deployment
SonarQube or similar	Static analysis	Quality/security findings

3.3 Communications Interfaces

HTTPS shall be used for request/response traffic. WSS shall be used in production. WebSocket messages shall use a versioned JSON event envelope containing event type, timestamp and payload.

4. Analysis Models

Principal actors: Guest, Member, Moderator and Community Admin. Detailed use cases and data model are provided in the Design Specification.

5. System Features

Identity & Profile

FR-AUTH-01 [High]: The system shall allow a guest to register using a unique username/email and password.

FR-AUTH-02 [High]: The system shall authenticate valid credentials and reject invalid credentials.

FR-AUTH-03 [High]: The system shall allow logout and invalidate the active session.

FR-AUTH-04 [Medium]: The system shall allow a user to view and update permitted profile fields.

Community & Channel Management

FR-COM-01 [High]: A member shall be able to create a community with a unique name for that owner.

FR-COM-02 [High]: A community admin shall be able to invite users and manage membership.

FR-COM-03 [High]: A community admin shall be able to create, rename and delete text channels subject to authorization.

FR-COM-04 [High]: The system shall enforce role/channel permissions before protected actions.

Real-Time Messaging & DMs

FR-MSG-01 [High]: A member shall be able to send a message to an authorized text channel.

FR-MSG-02 [High]: A sent channel message shall be delivered to connected authorized clients through WebSockets.

FR-MSG-03 [High]: A member shall be able to send and receive one-to-one direct messages.

FR-MSG-04 [Medium]: The system shall support message edit and delete actions subject to authorization.

FR-MSG-05 [Medium]: The system shall expose typing and presence events to authorized peers.

Reactions, Search & Notifications

FR-UX-01 [Medium]: A member shall be able to add or remove a reaction from an accessible message.

FR-UX-02 [Medium]: A member shall be able to search messages in communities they are authorized to access.

FR-UX-03 [Medium]: The system shall maintain unread counts and notifications for relevant new messages.

Moderation & Audit

FR-MOD-01 [High]: A moderator/admin shall be able to remove an offending message.

FR-MOD-02 [High]: A moderator/admin shall be able to restrict or remove a member according to community permissions.

FR-MOD-03 [High]: The system shall record security-relevant moderation events in an audit trail.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- WebSocket message acknowledgement target: ≤500 ms p95 under the defined test load in the test environment.
- Core UI/API operation target: ≤2 s under normal test conditions.
- Search response target: ≤2 s for the defined evaluation dataset.

6.2 Safety Requirements

The system is not safety-critical. Destructive community-level actions require authorization and confirmation.

6.3 Security Requirements

- Passwords never stored in plaintext
- Protected endpoints require authentication and authorization
- Server-side permission checks
- Safe rendering and input validation
- Rate limiting/abuse controls
- Security-relevant audit events
- Secrets supplied through environment/secret configuration

6.4 Software Quality Attributes

Attribute	Target
Maintainability	Modular API/WebSocket/persistence/UI separation
Testability	Unit tests for core services; integration tests for API/DB/socket flows
Reliability	Graceful reconnect/error handling without message corruption
Usability	Core chat actions discoverable without documentation
Portability	Reproducible Docker deployment
Security	No unresolved critical security findings at release

6.5 Business and Domain Rules

- Only members access private community content.
- Only admins change community roles and protected channel permissions.
- Moderators act only within communities where they have permission.
- Users edit/delete their own messages unless moderation permission applies.
- Deleted messages are not visible in normal user views.

7. Other Requirements

- Database shall enforce keys and uniqueness constraints.
- Timestamps stored consistently.
- Logs shall support debugging without passwords or secrets.

Appendix A: Glossary

Term	Definition
Community/Server	Persistent group space containing members, roles and channels.
Channel	Text conversation space within a community.
DM	Direct message conversation between two users.
WebSocket	Persistent bidirectional real-time communication channel.
Presence	Availability state such as online, idle or offline.
Role	Permission grouping within a community.
MVP	Minimum viable product.
RTM	Requirement Traceability Matrix.

Appendix B: Field Layouts

Detailed field-level specification: 08_Field_Layouts.xlsx.

Appendix C: Requirement Traceability Matrix

Initial RTM: 07_Requirement_Traceability_Matrix.xlsx.